
ОРГАНІЗАЦІЯ БАЗ ДАНИХ ТА ЗНАНЬ

PostgreSQL. Операції з даними.

Додавання даних. Команда Insert.

Для додавання даних застосовується команда INSERT , яка має наступний формальний синтаксис:

```
1 INSERT INTO имя_таблицы (столбец1, столбец2, ... столбецN)
2 VALUES (значение1, значение2, ... значениеN)
```

Після **INSERT INTO** йде ім'я таблиці, потім в дужках вказуються всі стовпці через кому, в які треба додавати дані. І в кінці після слова **VALUES** в дужках перераховуються значення, що додаються.

Припустимо, у нас в базі даних є наступна таблиця:

```
1 CREATE TABLE Products
2 (
3     Id SERIAL PRIMARY KEY,
4     ProductName VARCHAR(30) NOT NULL,
5     Manufacturer VARCHAR(20) NOT NULL,
6     ProductCount INTEGER DEFAULT 0,
7     Price NUMERIC
8 );
```

Додамо в неї один рядок за допомогою команди INSERT:

```
1 INSERT INTO Products VALUES (1, 'Galaxy S9', 'Samsung', 4, 63000)
```

Також при введенні значень можна вказати безпосередні стовпці, в які будуть додаватися значення:

```
1 INSERT INTO Products (ProductName, Price, Manufacturer)
2 VALUES ('iPhone X', 71000, 'Apple');
```

Також ми можемо додати відразу кілька рядків:

```
1 INSERT INTO Products (ProductName, Manufacturer, Price, ProductCount)
2 VALUES
3 ('iPhone 6', 'Apple', 3, 36000),
4 ('Galaxy S8', 'Samsung', 2, 46000),
5 ('Galaxy S8 Plus', 'Samsung', 1, 56000)
```

В даному випадку в таблицю будуть додані три рядки.

Повернення значень

Якщо ми додаємо значення тільки для частини стовпців, то ми можемо не знати, які значення будуть у інших стовпців. Наприклад, яке значення отримає стовпець `id` у таблиці `Товари`. За допомогою оператора **RETURNING** ми можемо отримати це значення:

```
1 INSERT INTO Products
2 (ProductName, Manufacturer, Price, ProductCount)
3 VALUES('Desire 12', 'HTC', 8, 21000) RETURNING id;
```

1 INSERT INTO Products
2 (ProductName, Manufacturer, Price, ProductCount)
3 VALUES('Desire 12', 'HTC', 8, 21000) RETURNING id;

Data Output Explain Messages Query History

	id
	integer
1	6

RETURNING - це операція в SQL, яка використовується для повернення значень стовпців після вставки, оновлення або видалення даних з таблиці. В PostgreSQL ця операція зазвичай використовується разом з INSERT, UPDATE або DELETE запитами.

Ось приклади реалізації RETURNING на PostgreSQL:

RETURNING з INSERT запитом:

```
-- Вставка нового рядка в таблицю "users" і отримання повернених значень
INSERT INTO users (username, email, registration_date)
VALUES ('newuser', 'newuser@example.com', NOW())
RETURNING user_id, username, registration_date;
```

RETURNING з UPDATE запитом:

```
-- Оновлення інформації для користувача з id=1 і отримання повернених значень
UPDATE users
SET email = 'updated@example.com'
WHERE user_id = 1
RETURNING user_id, username, email;
```

RETURNING з DELETE запитом:

```
-- Видалення користувача з id=2 і отримання повернених значень перед видаленням
DELETE FROM users
WHERE user_id = 2
RETURNING user_id, username, email;
```

Отримання даних. Команда Select.

Для отримання даних з БД застосовується команда SELECT. У спрощеному вигляді вона має наступний синтаксис:

```
1 SELECT список_столбцов FROM имя_таблицы
```

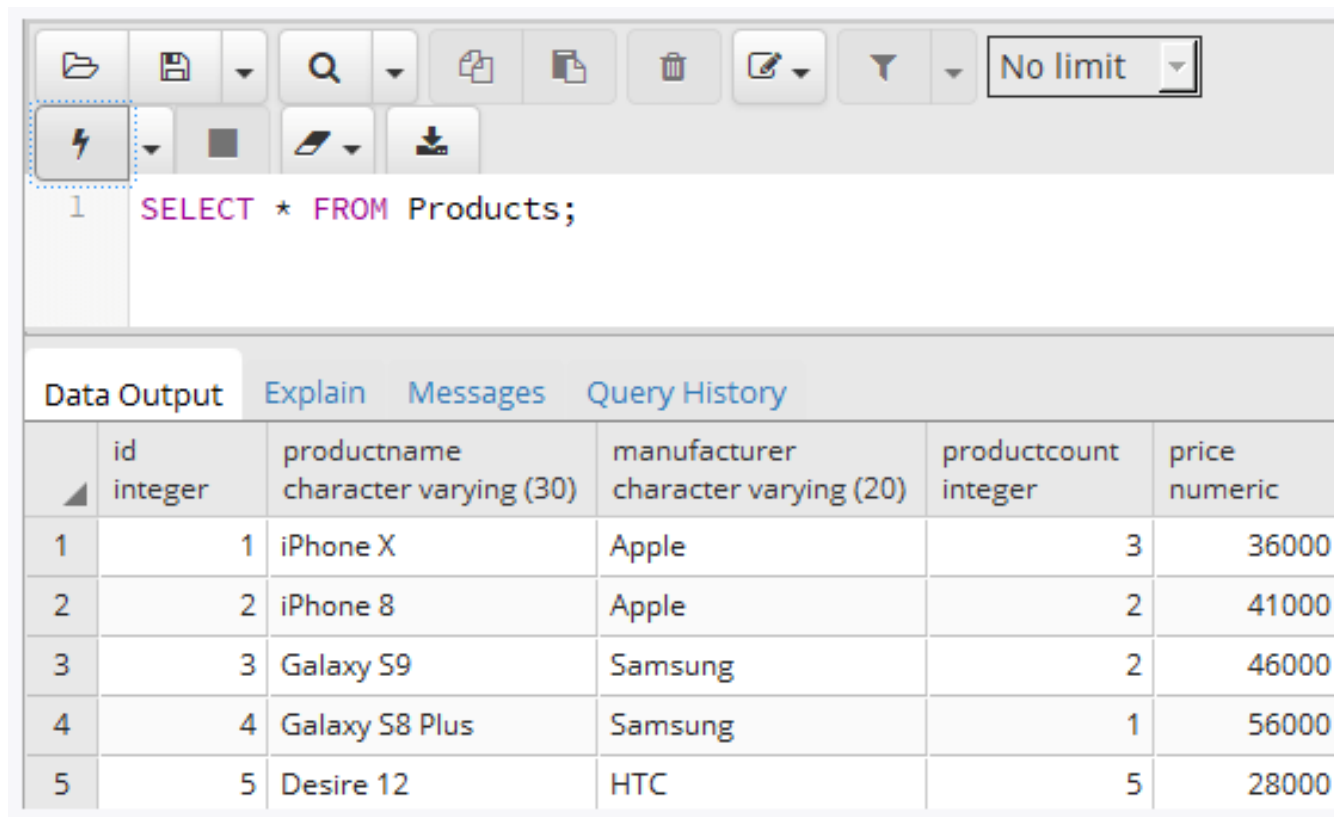
Наприклад, нехай раніше була створена таблиця Products, і в неї додані деякі початкові дані:

```
1 CREATE TABLE Products
2 (
3     Id SERIAL PRIMARY KEY,
4     ProductName VARCHAR(30) NOT NULL,
5     Manufacturer VARCHAR(20) NOT NULL,
6     ProductCount INTEGER DEFAULT 0,
7     Price NUMERIC
8 );
9
10 INSERT INTO Products (ProductName, Manufacturer, ProductCount, Price)
11 VALUES
12 ('iPhone X', 'Apple', 3, 36000),
13 ('iPhone 8', 'Apple', 2, 41000),
14 ('Galaxy S9', 'Samsung', 2, 46000),
15 ('Galaxy S8 Plus', 'Samsung', 1, 56000),
16 ('Desire 12', 'HTC', 5, 28000);
```

Отримаємо всі об'єкти з цієї таблиці:

```
1 SELECT * FROM Products;
```

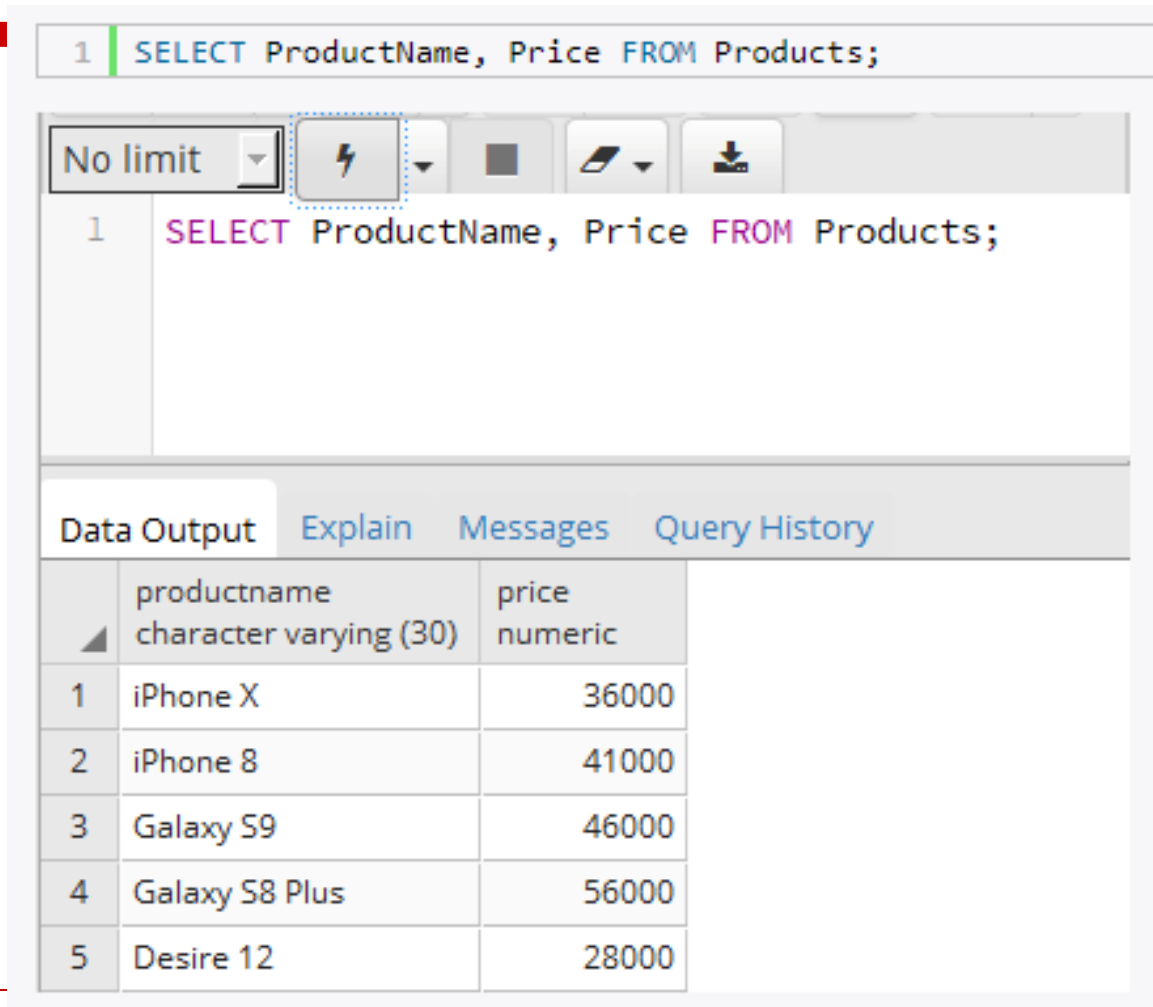
Символ зірочка * вказує, що нам треба отримати всі стовпці.



The screenshot shows a database query interface. At the top, there is a toolbar with various icons for file operations, search, and execution. Below the toolbar, the SQL query `1 SELECT * FROM Products;` is entered. The interface has tabs for **Data Output**, **Explain**, **Messages**, and **Query History**. The **Data Output** tab is active, displaying a table with the results of the query. The table has five columns: **id** (integer), **productname** (character varying (30)), **manufacturer** (character varying (20)), **productcount** (integer), and **price** (numeric). The results show five rows of data for different products.

	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	1	iPhone X	Apple	3	36000
2	2	iPhone 8	Apple	2	41000
3	3	Galaxy S9	Samsung	2	46000
4	4	Galaxy S8 Plus	Samsung	1	56000
5	5	Desire 12	HTC	5	28000

Якщо нам треба отримати дані не по всіх, а по якимось конкретним стовпцями, то тоді всі ці специфікації стовпців перераховуються через кому після **SELECT**:



The screenshot shows a SQL query editor interface. At the top, a text box contains the query: `1 SELECT ProductName, Price FROM Products;`. Below the text box is a toolbar with icons for 'No limit', a lightning bolt (highlighted with a dashed blue box), a dropdown arrow, a square, a document, and a download icon. Below the toolbar, the same query is displayed in a code editor: `1 SELECT ProductName, Price FROM Products;`. At the bottom, there is a tabbed interface with 'Data Output' selected. Below the tabs is a table with the following data:

	productname character varying (30)	price numeric
1	iPhone X	36000
2	iPhone 8	41000
3	Galaxy S9	46000
4	Galaxy S8 Plus	56000
5	Desire 12	28000

Специфікація стовпця необов'язково повинна представляти його назву. Це може бути будь-який вираз, наприклад, результат арифметичної операції. Так, виконаємо наступний запит:

```
1 SELECT ProductCount, Manufacturer, Price * ProductCount
2 FROM Products;
```

За допомогою оператора **AS** можна змінити назву вихідного стовпця або визначити його псевдонім:

```
1 SELECT ProductCount AS Title,  
2 Manufacturer,  
3 Price * ProductCount AS TotalSum  
4 FROM Products;
```

В даному випадку результатом вибірки є дані по 3-м стовпцях. Для першого стовпця визначається псевдонім Title, хоча в реальності він буде представляти стовпець ProductName. Другий стовпець зберігає свою назву - Manufacturer. Третій стовпець TotalSum зберігає добуток стовпців ProductCount і Price.

```
1 SELECT ProductCount AS Title,  
2 Manufacturer,  
3 Price * ProductCount AS TotalSum  
4 FROM Products;
```

Data Output

[Explain](#)

[Messages](#)

[Query History](#)

	title integer	manufacturer character varying (20)	totalsum numeric
1	3	Apple	108000
2	2	Apple	82000
3	2	Samsung	92000
4	1	Samsung	56000
5	5	HTC	140000

Фільтрація. WHERE.

Для фільтрації даних застосовується оператор WHERE , після якого вказується умова, на підставі якої проводиться фільтрація:

```
1 | WHERE условие
```

Якщо умова істинна, то рядок потрапляє в результуючу вибірку. У PostgreSQL можна застосовувати такі операції порівняння:

= : Порівняння на рівність

<> : Порівняння на нерівність

< : Менше ніж

> : Більше ніж

!< : Не менш ніж

!> : Але лише

<= : Менше ніж або дорівнює

>= : Більше ніж або дорівнює

Наприклад, знайдемо всіх товари, виробником яких є компанія Apple:

```
1 SELECT * FROM Products
2 WHERE Manufacturer = 'Apple';
```

```
1 SELECT * FROM Products
2 WHERE Manufacturer = 'Apple';
```

Data Output Explain Messages Query History					
	id	productname	manufacturer	productcount	price
	integer	character varying (30)	character varying (20)	integer	numeric
1	1	iPhone X	Apple	3	36000
2	2	iPhone 8	Apple	2	41000

Інший приклад - знайдемо всі товари, у яких ціна менше 29000:

```
1 SELECT * FROM Products
2 WHERE Price < 39000;
```

Як умова можуть використовуватися і більш складні вирази. Наприклад, знайдемо всі товари, у яких сукупна вартість понад 90 000:

```
1 SELECT * FROM Products
2 WHERE Price * ProductCount > 90000;
```

Data Output Explain Messages Query History					
	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	1	iPhone X	Apple	3	36000
2	3	Galaxy S9	Samsung	2	46000
3	5	Desire 12	HTC	5	28000

Логічні оператори

Щоб об'єднати кілька умов в одну, в PostgreSQL можна використовувати логічні оператори:

AND : операція логічного І. Вона об'єднує два вирази:

```
1 | выражение1 AND выражение2
```

Тільки якщо обидва ці вирази одночасно істинні, то і загальні умова оператора AND також було це слово. Тобто якщо і перша умова істинна, і друга істинна.

Наприклад, виберемо усі товари, у яких виробник Samsung і одночасно ціна більше 50000:

```
1 SELECT * FROM Products
2 WHERE Manufacturer = 'Samsung' AND Price > 50000;
```

```
1 SELECT * FROM Products
2 WHERE Manufacturer = 'Samsung' AND Price > 50000;
```

Data Output	Explain	Messages	Query History		
	id	productname	manufacturer	productcount	price
	integer	character varying (30)	character varying (20)	integer	numeric
1	4	Galaxy S8 Plus	Samsung	1	56000

OR : операція логічного АБО. Вона також об'єднує два вирази:

```
1 | вираження1 OR вираження2
```

Якщо хоча б один з цих виразів істинний, то загальна умова оператора OR також було це слово. Тобто якщо або перша умова істинна, або друга істинна.

Тепер змінимо оператор на OR. Тобто виберемо усі товари, у яких або виробник Samsung, або ціна більше 50000:

```
1 SELECT * FROM Products
2 WHERE Manufacturer = 'Samsung' OR Price > 50000;
```

```
1 SELECT * FROM Products
2 WHERE Manufacturer = 'Samsung' OR Price > 50000;|
```

Data Output

[Explain](#)

[Messages](#)

[Query History](#)

	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	3	Galaxy S9	Samsung	2	46000
2	4	Galaxy S8 Plus	Samsung	1	56000

NOT : операція логічного заперечення. Якщо вираз в цій операції хибний, то загальна умова істинна.

1	NOT виражение
---	---------------

Застосування оператора NOT - виберемо усі товари, у яких виробник не Samsung:

```

1 SELECT * FROM Products
2 WHERE NOT Manufacturer = 'Samsung';

```

```

1 SELECT * FROM Products
2 WHERE NOT Manufacturer = 'Samsung';|

```

Data Output	Explain	Messages	Query History		
id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric	
1	1	iPhone X	Apple	3	36000
2	2	iPhone 8	Apple	2	41000
3	5	Desire 12	HTC	5	28000

Але в більшості випадків цілком можна обійтися без оператора NOT. Так, попередній приклад ми можемо переписати таким чином:

```
1 SELECT * FROM Products
2 WHERE Manufacturer <> 'Samsung'
```

Також в одній команді **SELECT** можна використовувати відразу кілька операторів:

```
1 SELECT * FROM Products  
2 WHERE Manufacturer = 'Samsung' OR Price > 30000 AND ProductCount > 2;
```

Так як оператор AND має більш високий пріоритет, то спочатку буде виконуватися підвираз Price > 30000 AND ProductCount > 2 , і тільки потім оператор OR. Тобто тут вибираються товари, яких на складі більше 2 і у яких одночасно ціна більше 30000, або ті товари, виробником яких є Samsung.

За допомогою дужок ми також можемо перевизначити порядок операцій:

```
1 SELECT * FROM Products
2 WHERE (Manufacturer = 'Samsung' OR Price > 30000) AND ProductCount > 2;
```

IS NULL

Ряд стовпців може допускати значення **NULL**. Це значення не еквівалентне порожньому рядку. NULL представляє повну відсутність будь-якого значення. І для перевірки на наявність подібного значення застосовується оператор **IS NULL**.

Наприклад, виберемо усі товари, у яких не встановлено поле ProductCount:

```
1 SELECT * FROM Products
2 WHERE ProductCount IS NULL;
```

Якщо, навпаки, необхідно отримати рядки, у яких поле ProductCount не дорівнює NULL, то можна використовувати оператор NOT:

```
1 SELECT * FROM Products
2 WHERE ProductCount IS NOT NULL;
```

Команда UPDATE

Для поновлення даних в базі даних PostgreSQL застосовується команда UPDATE . Вона має наступний загальний формальний синтаксис:

```
1 UPDATE имя_таблицы
2 SET столбец1 = значение1, столбец2 = значение2, ... столбецN = значениеN
3 [WHERE условие_обновления]
```

```
1 SELECT * FROM Products;
```

Data Output Explain Messages Query History

	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	1	iPhone X	Apple	3	36000
2	2	iPhone 8	Apple	2	41000
3	3	Galaxy S9	Samsung	2	46000
4	4	Galaxy S8 Plus	Samsung	1	56000
5	5	Desire 12	HTC	5	28000

Наприклад, збільшимо у всіх товарів ціну на 3000. Оновимо таблицю.

```
1 UPDATE Products
2 SET Price = Price + 3000;
3
4 SELECT * FROM Products;
```

Data Output

[Explain](#)

[Messages](#)

[Query History](#)

	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	1	iPhone X	Apple	3	39000
2	2	iPhone 8	Apple	2	44000
3	3	Galaxy S9	Samsung	2	49000
4	4	Galaxy S8 Plus	Samsung	1	59000
5	5	Desire 12	HTC	5	31000

В даному випадку оновлення стосується всіх рядків. За допомогою виразу **WHERE** можна за допомогою умові конкретизувати оновлювані рядки - якщо рядок відповідає умові, то вона буде оновлюватися.

Наприклад, змінимо назву виробника з "Samsung" на "Samsung Inc.":

```
1 UPDATE Products
2 SET Manufacturer = 'Samsung Inc.'
3 WHERE Manufacturer = 'Samsung';
4
5 SELECT * FROM Products;
```

Data Output Explain Messages Query History

	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	1	iPhone X	Apple	3	39000
2	2	iPhone 8	Apple	2	44000
3	5	Desire 12	HTC	5	31000
4	3	Galaxy S9	Samsung Inc.	2	49000
5	4	Galaxy S8 Plus	Samsung Inc.	1	59000

Також можна оновлювати відразу декілька
стовпців:

```
1 UPDATE Products
2 SET Manufacturer = 'Samsung',
3     ProductCount = ProductCount + 3
4 WHERE Manufacturer = 'Samsung Inc.';
```

Припустимо, що у нас є таблиця "users" з такими стовпцями: user_id, username, email, і ми хочемо оновити інформацію користувача з певним user_id.

```
-- Приклад оновлення інформації для користувача з id=1
UPDATE users
SET username = 'новеІм'я', email = 'новий@email.com'
WHERE user_id = 1;
```

У цьому прикладі ми використовуємо оператор UPDATE для оновлення імені користувача (username) та електронної пошти (email) для користувача з user_id = 1. У рядку WHERE вказано умову, за якою рядок повинен бути оновлений.

Після виконання цього запиту, дані вказаного користувача будуть оновлені в таблиці "users". Важливо враховувати, що WHERE умова визначає, який саме рядок буде оновлений, тому вона має бути налаштована на відповідний рядок або набір рядків для оновлення.

Видалення даних. Команда DELETE.

Для видалення даних в PostgreSQL застосовується команда DELETE . Вона має наступний синтаксис:

```
1 DELETE FROM имя_таблицы
2 [WHERE условие_удаления]
```

Наприклад, видалимо рядки, у яких виробник - Apple:

```
1 DELETE FROM Products
2 WHERE Manufacturer='Apple';
```

```
1 SELECT * FROM Products;
```

Data Output Explain Messages Query History					
	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	1	iPhone X	Apple	3	39000
2	2	iPhone 8	Apple	2	44000
3	5	Desire 12	HTC	5	31000
4	3	Galaxy S9	Samsung	5	49000
5	4	Galaxy S8 Plus	Samsung	4	59000

Результат:

```
1 DELETE FROM Products
2 WHERE Manufacturer='Apple';
3
4 SELECT * FROM Products;
```

Data Output

[Explain](#)

[Messages](#)

[Query History](#)

	id integer	productname character varying (30)	manufacturer character varying (20)	productcount integer	price numeric
1	5	Desire 12	HTC	5	31000
2	3	Galaxy S9	Samsung	5	49000
3	4	Galaxy S8 Plus	Samsung	4	59000

Або видалимо всі товари, виробником яких є HTC і які мають ціну менше 35000:

```
1 DELETE FROM Products
2 WHERE Manufacturer='HTC' AND Price < 15000;
```

Якщо необхідно зовсім видалити всі рядки незалежно від умови, то умову можна не вказувати:

```
1 DELETE FROM Products;
```


Приклад

Предметна область “Бронювання квитків в готелі”

Сутність "Клієнт":

- Ім'я (перше і останнє)
- Контактний номер телефону
- Адреса електронної пошти
- Адреса проживання

Сутність "Готель":

- Назва готелю
- Адреса готелю
- Кількість кімнат
- Опис готелю
- Зручності (басейн, ресторан, Wi-Fi і т. д.)

Сутність "Кімната":

- Номер кімнати
 - Тип кімнати (одномісна, двомісна, люкс і т. д.)
 - Вартість проживання за ніч
 - Кількість ліжок та їх тип (односпальне, двоспальне і т. д.)
 - Додаткові зручності (телевізор, кондиціонер, ванна кімната і т. д.)
-

Сутність "Бронювання":

- Дата початку бронювання
- Дата закінчення бронювання
- Кількість дорослих гостей
- Кількість дітей
- Статус бронювання (підтверджено, в очікуванні, скасовано і т. д.)

Сутність "Платіж":

- Спосіб оплати (готівка, кредитна карта, переказ і т. д.)
- Сума оплати
- Дата оплати

Сутність "Персонал готелю":

- Ім'я та прізвище співробітника
 - Посада (адміністратор, прибиральник, офіціант і т. д.)
 - Години роботи та графік роботи
-

Запит на створення таблиці "clients":

```
CREATE TABLE clients (  
    client_id SERIAL PRIMARY KEY,  
    first_name VARCHAR(50) NOT NULL,  
    last_name VARCHAR(50) NOT NULL,  
    phone_number VARCHAR(20),  
    email VARCHAR(100),  
    address VARCHAR(255)  
);
```

Запит на створення таблиці "hotels":

```
CREATE TABLE hotels (  
    hotel_id SERIAL PRIMARY KEY,  
    hotel_name VARCHAR(100) NOT NULL,  
    address VARCHAR(255) NOT NULL,  
    room_count INT NOT NULL,  
    description TEXT,  
    amenities TEXT[]  
);
```

Запит на створення таблиці "rooms":

```
CREATE TABLE rooms (  
    room_id SERIAL PRIMARY KEY,  
    room_number VARCHAR(10) NOT NULL,  
    room_type VARCHAR(50) NOT NULL,  
    price NUMERIC(10, 2) NOT NULL,  
    bed_count INT NOT NULL,  
    bed_type VARCHAR(50),  
    amenities TEXT[]  
);
```

Запит на створення таблиці "bookings":

```
CREATE TABLE bookings (  
    booking_id SERIAL PRIMARY KEY,  
    client_id INT NOT NULL,  
    room_id INT NOT NULL,  
    start_date DATE NOT NULL,  
    end_date DATE NOT NULL,  
    adults INT NOT NULL,  
    children INT NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    FOREIGN KEY (client_id) REFERENCES clients (client_id),  
    FOREIGN KEY (room_id) REFERENCES rooms (room_id)  
);
```

Запит на створення таблиці "payments":

```
CREATE TABLE payments (  
    payment_id SERIAL PRIMARY KEY,  
    booking_id INT NOT NULL,  
    payment_method VARCHAR(50) NOT NULL,  
    amount NUMERIC(10, 2) NOT NULL,  
    payment_date DATE NOT NULL,  
    FOREIGN KEY (booking_id) REFERENCES bookings (booking_id)  
);
```

```
-- Додавання записів до таблиці "clients"
INSERT INTO clients (first_name, last_name, phone_number, email, address)
VALUES ('John', 'Doe', '123-456-7890', 'john.doe@example.com', '123 Main St')

-- Додавання записів до таблиці "hotels"
INSERT INTO hotels (hotel_name, address, room_count, description, amenities)
VALUES ('Sample Hotel', '456 Elm St', 50, 'A beautiful hotel in the city center', 'Free Wi-Fi, Pool, Gym')

-- Додавання записів до таблиці "rooms"
INSERT INTO rooms (room_number, room_type, price, bed_count, bed_type, amenities)
VALUES ('101', 'Single', 75.00, 1, 'Single', ARRAY['TV', 'Air conditioning']),
       ('102', 'Double', 100.00, 2, 'Double', ARRAY['TV', 'Air conditioning']),
       ('103', 'Suite', 150.00, 2, 'King-size', ARRAY['TV', 'Air conditioning'])

-- Додавання записів до таблиці "bookings"
INSERT INTO bookings (client_id, room_id, start_date, end_date, adults, children, status)
VALUES (1, 1, '2023-09-20', '2023-09-25', 1, 0, 'Confirmed'),
       (1, 2, '2023-09-22', '2023-09-24', 2, 1, 'Pending');

-- Додавання записів до таблиці "payments"
INSERT INTO payments (booking_id, payment_method, amount, payment_date)
VALUES (1, 'Credit Card', 375.00, '2023-09-20'),
       (2, 'Cash', 200.00, '2023-09-22');
```



```
SELECT * FROM rooms
WHERE price < 100.00;
```

room_id	room_number	room_type	price	bed_count	bed_type	amenities
1	101	Single	75.00	1	Single	{TV,Air
2	102	Double	100.00	2	Double	{TV,Air

```
SELECT * FROM bookings
WHERE status <> 'Confirmed';
```

client_id	room_id	start_date	end_date	adults	children	status
1	2	2023-09-22	2023-09-24	2	1	Pending

```
SELECT * FROM bookings
WHERE adults < 2;
```

booking_id	client_id	room_id	start_date	end_date	adults	children
1	1	1	2023-09-20	2023-09-25	1	0
2	1	2	2023-09-22	2023-09-24	2	1

```
SELECT * FROM clients
WHERE client_id = 1;
```

client_id	first_name	last_name	phone_number	email
1	John	Doe	123-456-7890	john.doe@example.com